

QA Execution & Testing Guide — Forecasting Service API

Architecture Generalization Notice:

- All legacy hardcoded configurations, hardcoded ICP branch paths, and environment variable path overrides (e.g. `DATA_PATH` env overrides) have been **COMPLETELY REMOVED**.
- The forecasting engine is now 100% dynamic: it auto-discovers all branches, transaction categories, and dataset files directly from the root `Data` folder, making it fully client-agnostic and ready for any organization.

1. Zip Extraction & Directory Setup (Windows)

Follow these exact steps from the moment you receive the project ZIP file:

Step 1: Extract the ZIP File

1. Receive `forecasting-service-api.zip` (or project archive).
2. Right-click the `.zip` file → Click **Extract All...**
3. Extract the contents to your working folder (e.g. `C:\forecasting-service-api`).

Step 2: Open Windows PowerShell

1. Press `Win + R` on your keyboard to open the **Run** dialog.
2. Type `powershell` and press **Enter**.

Step 3: Navigate to the `backend` Directory

In the PowerShell window, type the following `cd` command to enter the `backend` folder:

```
cd C:\forecasting-service-api\backend
```

(Make sure your terminal prompt reads `PS C:\forecasting-service-api\backend>` before running any scripts!)

2. File Placement & Data Format Requirements

Flat Directory Layout (No Nested Subfolders)

- **No Nested Subfolders Required:** You do **NOT** need to create year subfolders (e.g. `Data\2026\`) or nested directories (e.g. `Data\icp_data\`).
- Simply dump your raw `.csv`, `.xlsx`, or `.parquet` files directly into the root `backend\Data` folder:
`C:\forecasting-service-api\Data\Daily Ticket Log July 2026.csv`

Supported File Formats

- **CSV Files:** `.csv` (Auto-converted to `.parquet` upon script execution)
- **Excel Files:** `.xlsx` , `.xls` (Auto-converted to `.parquet` upon script execution)
- **Parquet Files:** `.parquet`

File Naming Convention & Data Integrity Rules (CRITICAL)

Rule 1 — Mandatory File Naming Pattern:

All input CSV / Excel files must follow the descriptive month/year naming convention:

- `Daily Ticket Log July 2026.csv`
- `Daily Ticket Log August 2026.csv`
- `Daily Ticket Log September 2026.csv`

Rule 2 — NEVER Append Data to an Already-Ingested File:

- The database tracks ingested file names (`IngestedFile` table) to prevent duplicate row ingestion.
- If you add new data rows to a file that was already processed (e.g. appending August data inside `Daily Ticket Log July 2026.csv`), the system will **SKIP** the file because `Daily Ticket Log July 2026.csv` is marked as already processed.
- **Every new month's data MUST be saved as a NEW file with a unique filename** (e.g. `Daily Ticket Log August 2026.csv`).

Rule 3 — Do NOT Copy/Paste Rows to Create Fake Datasets:

- Please do **NOT** attempt to copy and paste historical rows N times to artificially enlarge a dataset.
- The pipeline aggregates daily volume by date (`COUNT(rows)` per `Issue Date`). Copying identical dates/rows simply multiplies a single day's count rather than providing real multi-day variance or weekly seasonality.
- For realistic testing, ensure each dataset file contains genuine daily records across actual calendar dates.

Mandatory Column Names

Your input CSV, Excel, or Parquet file **must** include the following standard columns:

Column Name	Description / Example	Mandatory?
<code>Issue Date</code>	Date ticket was created (<code>YYYY-MM-DD</code> or <code>2026-07-01</code>)	Yes

Column Name	Description / Example	Mandatory?
Branch Name	Service branch location name (e.g. Brampton)	Yes
Category Name	Transaction/service category (e.g. A Road Test)	Yes
Region Name	Operational region/province (e.g. Ontario)	Yes
Ticket Id	Unique ticket ID integer	Optional
Wait Time	Service wait time in seconds	Optional
Service Time	Transaction handling time in seconds	Optional

3. Environment & Database Setup (.env)

Before running tests, ensure the environment file `.env` is present inside `C:\forecasting-service-api\backend\.env`.

Option A: Microsoft SQL Server Configuration (Production / Remote Testing)

To connect to Microsoft SQL Server, configure `C:\forecasting-service-api\backend\.env` with your SQL Server credentials:

```
DB_SERVER=localhost
DB_PORT=1433
DB_USER=SA
DB_PASSWORD=YourStrong@Passw0rd
DB_NAME=forecast_app
```

Option B: Local SQLite Configuration (Local Offline Testing)

If `.env` is omitted or MSSQL credentials are not supplied, the system **automatically falls back** to a local SQLite database (`forecast_app.db`) in the `backend` folder — zero database setup required!

4. How to Run Initial Setup & Pipeline (Windows)

Once you are inside `PS C:\forecasting-service-api\backend>`, execute **one single command**:

```
.\start.ps1
```

(Note: If execution policy blocks script execution, run PowerShell once as Administrator: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`)

What `start.ps1` Does Automatically (100% Hands-Off):

- 1. **Creates/Reuses Virtual Environment:** Builds or activates `venv` safely without wiping installed packages.
- 2. **Installs Dependencies:** Automatically installs/updates all required packages from `requirements.txt` (including `openpyxl`).
- 3. **Database Initialization:** Auto-creates all missing MSSQL/SQLite database tables (`init_db()`).
- 4. **CSV/Excel Auto-Conversion & Ingestion:** Converts `.csv` & `.xlsx` → `.parquet` and ingests actuals into `actual_traffic` .
- 5. **Model Retraining & Forecast Generation:** Retrains Prophet / Cold-Start Engine and saves 365 daily predictions to `daily_forecasts` .
- 6. **Launches Backend Server:** Starts FastAPI server on `http://localhost:8000` .

5. How to Run Subsequent / Next Training Cycles (Adding New Months)

When testing continuous learning as new months of data arrive:

- 1. Copy the new month's CSV file alongside your previous files in `C:\forecasting-service-api\Data\` (e.g. `Daily Ticket Log August 2026.csv`).
- 2. If the backend server is currently running in PowerShell, press `Ctrl + C` to stop the server.
- 3. In PowerShell (`PS C:\forecasting-service-api\backend>`), run:
`powershell .\start.ps1`

What Happens Automatically:

- **Idempotent Ingestion:** Auto-detects `Daily Ticket Log August 2026.csv` , converts it, and ingests ONLY the new August data into SQL Server without duplicating July.
- **Cumulative Retraining:** Combines July + August history to retrain Prophet on cumulative days.
- **Horizon Shift:** Automatically shifts the 365-day prediction start date forward to **September 1, 2026** → **August 31, 2027**.

6. What to Expect during Testing (Cold-Start vs Matured Data)

Dataset History	Triggered Engine	Expected Forecast Behavior	Terminal Log Indicator
1 to 6 Days (Cold-Start)	Cold-Start Heuristic Baseline	Generates 365-day forecasts immediately. Weekday ≈ 100%, Weekend ≈ 20%.	Cold-Start Triggered for Branch...
7 to 359 Days (Short History)	Prophet ML (Yearly Seasonality OFF)	Fits 7-day weekly waves. Yearly seasonality auto-disabled to prevent negative divergence.	Training: Branch (Key: Branch)...
360+ Days (Full 1 Year)	Prophet ML (Yearly Seasonality ON)	Multiplicative yearly seasonality auto-enabled. Achieves peak precision (≈ 99% accuracy).	Training on rolling 3-year window...

7. Master 6-Scenario Progression Benchmark Table

The following ground-zero benchmark results demonstrate how model accuracy and stability evolve progressively as data history expands from Day 1 to 1 Full Year:

Metric	Scenario 1 (1 Day)	Scenario 2 (7 Days)	Scenario 3 (31 Days)	Scenario 4 (62 Days)	Scenario 5 (153 Days)	Scenario 6 (365 Days / 1 Year)
Engine Used	Cold-Start Baseline	Prophet ML	Prophet ML	Prophet ML	Prophet ML	Prophet ML (Full Year)
Yearly Seasonality	OFF	OFF	OFF	OFF	OFF	AUTO-ENABLED (Multiplicative)
Input Data History	1 Day (Jul 1)	7 Days (Jul 1-7)	31 Days (Jul 1-31)	62 Days (Jul 1-Aug 31)	153 Days (Jul 1-Nov 30)	365 Days (Jul 1, 2026 → Jun 30, 2027)
Prediction Window	Jul 2, 26 → Jul 1, 27	Jul 8, 26 → Jul 7, 27	Aug 1, 26 → Jul 31, 27	Sep 1, 26 → Aug 31, 27	Dec 1, 26 → Nov 30, 27	Jul 1, 27 → Jun 29, 28
Immediate Weekday Avg	250 tickets/day	262 tickets/day	251–274 tickets/day	233–247 tickets/day	245–250 tickets/day	240–244 tickets/day (Target)
Immediate Weekend Avg	50 tickets/day	48 tickets/day	55 tickets/day	50–52 tickets/day	52 tickets/day	51 tickets/day (Target)
Month 6 Future Weekday	250 tickets/day	~12,368 tickets (drift)	~560 tickets (calming)	~237 tickets (stable)	~250–254 tickets	238–243 tickets/day (Perfect)
Month 6 Future Weekend	50 tickets/day	~1,017 tickets	~119 tickets	~48–50 tickets	~53 tickets	55–56 tickets/day (Perfect)
365-Day Weekday Avg	250.0 tickets/day	9,071.4 tickets/day	652.1 tickets/day	225.1 tickets/day	252.5 tickets/day	238.2 tickets/day (Target: ~240)
365-Day Weekend Avg	50.0 tickets/day	1,314.9 tickets/day	140.2 tickets/day	47.5 tickets/day	52.9 tickets/day	49.2 tickets/day (Target: ~50)
Model Precision	Cold-Start Fallback	Unstable (Slope Drift)	Moderate (~85%)	High (~92%)	Peak Precision (~98%)	PRODUCTION GOLD STANDARD (~99%+)

8. Verification & API Testing (Windows)

Once PowerShell prints Uvicorn running on http://0.0.0.0:8000 , QA can verify predictions via browser:

- 1. **Swagger UI Docs:** Open http://localhost:8000/docs in Google Chrome or Microsoft Edge.
- 2. **Monthly Forecast API:**
http://localhost:8000/api/forecast-by-month?month=8&year=2026&branch_name=Brampton
- 3. **Branch Categories API:**
http://localhost:8000/api/categories?branch_name=Brampton
- 4. **Historical Actuals API:**
http://localhost:8000/api/historical?branch_name=Brampton